SOFTWARE PROJECT MANAGEMENT

Mid-Term Exam Preparation Guide

Complete Notes | Tricks | Examples | Easy Summaries

Compiled from Lecture Slides • Ready for Exam Success

Table of Contents

01 **Introduction to Projects & SPM**
What is a project? Project vs Program, Project characteristics, IT projects

02 **Project Success & Failure**
CHAOS report, Software crisis, Classic mistakes, Success factors

03 **Software Project Management Fundamentals**
Boehm's metrics, PMI Knowledge Areas, Project life cycles, SDLCs

04 **Agile Project Management**
Agile overview, Scrum, Roles, Events, Artifacts, Selection criteria

05 **Process Management**
Process types, Choosing & tailoring a process, Project parameters, PM phases

06 **Initiating the Project**
Project charter, SOW, SMART goals, Stakeholders, Org structures

07 **Project Planning & WBS**
Planning steps, Scope management, WBS, Decomposition, Gantt, Requirements

01 — Introduction to Projects & Software Project Management

QUICK SUMMARY

- A PROJECT is a temporary, unique endeavor with defined scope, time, and cost.
- SOFTWARE PROJECT MANAGEMENT = define, plan, execute, and monitor software development.
- The TRIPLE CONSTRAINT: Scope ↔ Time ↔ Cost — balancing all three is the PM's core job.
- IT projects differ from normal projects due to complexity, tight schedules, and changing technology.

1.1 What Is a Project?

PROJECT (PMI)	A temporary endeavor undertaken to create a unique product or service.

A project is a sequence of connected and related activities (requirements, coding, testing, documentation) that must be completed by a specific time, within budget, and according to specification.

Key Characteristics of Every Project
✓ UNIQUE — Every project creates unique deliverables. Even "identical" projects differ due to context.
✓ TEMPORARY — Has a definite beginning AND end (via success, failure, or redundancy).
✓ CUSTOMER-SPECIFIED — Requirements come from the customer; meeting them = success.
✓ RESOURCE-CONSUMING — Uses people, physical, and monetary resources.
✓ 4 P's: Finite Time, People Assigned, Clear Roles, Things to Deliver.

EXAM TRICK: Remember the 4 P's of a project: PEOPLE, PRODUCT, PROCESS, PROJECT. Easy mnemonic: "4 P's make a Perfect Project Pass."

1.2 Project vs. Program

PROJECT	PROGRAM
Single endeavor with one goal	Collection of related projects
Shorter in duration	Longer in duration, broader scope
Managed by Project Manager	Managed by Program Manager
Example: Building Excel's spell-checker	Example: MS Office (Word + Excel + PowerPoint + ...)

EXAMPLE: Microsoft Office is a PROGRAM. Each component (Word, Excel, PowerPoint, Spelling) is a separate PROJECT within that program. The Program Manager oversees all of them.

1.3 What is Software Project Management (SPM)?

SPM	The art to define, plan, execute, and monitor the activities that will bring software products to existence.

Project managers strive to meet the triple constraint by balancing:

- Scope — What work needs to be done?
- Time — How long will it take?
- Cost — How much will it cost?

EXAM TRICK: "Fast, Cheap, Good — Choose Two." This is the classic Trade-off Triangle. In an exam, if asked which two constraints are fixed, the third is always flexible!

The Triple Constraint (Trade-off Triangle)
Q̄ Scope = All work required; no more, no less.
Time = Deadline; often fixed and non-negotiable.
ƽ Cost = Budget; needs careful estimation and control.
˚Changing one always affects at least one other!
˚Example: Add more features (scope) → need more time OR more money.

1.4 The 5 PM Process Groups

Project management always follows these five process groups:

1. Initiating/Define — Scope the project; charter it; identify stakeholders.
2. Planning — Develop the project plan. Collect requirements; identify schedule.
3. Executing — Launch the plan. Direct work; manage team; conduct procurements.
4. Monitoring & Controlling — Monitor and measure progress; take corrective actions.
5. Closing — Formalize acceptance; close project and procurements.

EXAM TRICK: Memorize the 5 groups with "I Plan Extremely Meticulous Closings": Initiating, Planning, Executing, Monitoring, Closing.

1.5 How Are IT/Software Projects Different?

Challenge	Description	Real-world Impact
Greater Complexity	Computing + communication + data = inherently complex	Hard to estimate and plan
Tight Schedules	"Rush to market" culture; deadlines are non-negotiable	Quality often suffers
Enterprise Dependency	Business success depends critically on IT	Failures are very costly
Rapidly Changing Tech	Technology can become obsolete during a project	Plans must be adaptive
Invisible Progress	Cannot see how far software has progressed (unlike a building)	Hard to measure & control
Custom & One-of-a-kind	Most software is written from scratch	No templates; high uncertainty

EXAMPLE: You can see 50% of a skyscraper built — it's obvious. But you cannot see 50% of a software system being built! This makes progress measurement a unique challenge in software projects.

1.6 Software Quality Principles

Software Quality Engineering Principle
"The quality of the software system is controlled by the quality of the PROCESS used to produce it."
Three pillars of Quality Management:
1. DOCUMENT the process — write it down so it's repeatable.
2. MEASURE the process — track metrics like defect rates, schedule variance.
3. IMPROVE the process — use measurement data to get better over time.

A well-defined process produces artifacts that are:

- Predictable — you know what you'll get
- Based on requirements — aligned with customer needs
- Easy to maintain — clear and consistent
- Of acceptable quality — within agreed standards
- Within milestones — on-time and on-budget
-

Three Levels of Software Process
METAPROCESS: Organization-level policies and strategies (long-term).
MACROPROCESS: Project-level policies for producing the complete software product.
MICROPROCESS: Team-level practices for achieving individual artifacts.

02 — Project Success & Failure

QUICK SUMMARY
- The CHAOS Report shows only ~26-39% of IT projects succeed fully.
- Classic failure reasons: bad requirements, lack of planning, weak personnel.
- Software failures cause BILLIONS in losses and real-world disasters.
- HealthCare.gov is the most detailed case study — know its lessons!
- 36 Classic Mistakes (AntiPatterns) grouped by: People, Process, Product, Technology.

2.1 The Software Crisis — Real Failures

Software project failures are extremely common and have caused major disasters:

- AT&T network crash (1990): 8-hour outage due to a missing "break" in a switch statement.
- Ariane 5 rocket explosion (1996): Exploded 37 seconds after launch due to a software integer overflow.
- NYSE shutdown (2001): Entire trading floor halted for over an hour due to a software problem.

EXAMPLE: The Ariane 5 explosion: Software reused from Ariane 4 could not handle the higher speed of Ariane 5. A number that was 64-bit was placed in a 16-bit space, causing overflow, erroneous steering commands, and a self-destruct. Cost: ~$500 million. Lesson: Never reuse software without understanding context differences.

2.2 The Standish Group CHAOS Reports

Metric	CHAOS 2001	CHAOS 2012
Successful projects (on-time, on-budget, full scope)	16-26%	39%
Challenged projects (late/over-budget/less features)	Large majority	43%
Failed/Cancelled projects	>31%	18%
Average schedule overrun	163%	Still significant
Average cost overrun	145%	Still significant
Delivered functionality vs. planned	67%	Improving

EXAM TRICK: In exams, remember: CHAOS 2012 → 39% success, 43% challenged, 18% cancelled. The trend is improving but still alarming! Use "39-43-18" as your memory anchor.

2.3 Case Study — HealthCare.gov (2013)

Background
The Affordable Care Act (ACA) was signed in March 2010.
HealthCare.gov launched October 1, 2013 as a one-stop health insurance marketplace.
CBO forecast: 7 million users in first year. Budget: $293M. Actual cost: >$500M then >$2B!

Problem Area	Detail
Performance	Landing page response time >8 seconds. "Annoyingly long wait times."
Stability	Intermittent crashes; availability ≈ 43% only!
Scalability	Could handle <1,100 concurrent users (50,000-60,000 expected)
Error Rate	≈6% errors per page — very high
Completion Rate	<30% of users could enroll successfully
Security	Final security tests not completed before launch

Root Causes — THREE categories:

Software Engineering Failures:

- Inadequate Testing: Full testing began only 2 weeks before launch!
- Pre-flight checklist (T-1 week): 41 of 91 functions FAILED.
- No end-to-end testing as late as 4 days before launch.
- Stress tests (T-1 day): Performance degraded with only 1,100 users.
- Evolving Requirements: Specs were delayed and modified repeatedly until summer 2013.
- No agile approach — committed to all-or-nothing launch date.

Management Failures:

- Substandard staffing: Only 10 developers on crucial part, only 1 was highly skilled.
- Management expertise was in getting contracts, not delivering them.
- Fragmented authority: Unclear responsibilities across 47+ contractors.

Government & Policy Failures:

- "Write-down-all-requirements-then-build" (Waterfall) approach — outdated.
- No-bid, cost-plus contracts rewarded contract-winning, not delivery excellence.

Lessons Learned from HealthCare.gov
Adopt Agile development — embrace change, test early and often.
Clear responsibility and accountability across all contractors.
Performance metrics and progress tracking from day one.

Integration testing and end-to-end testing are NON-NEGOTIABLE.

Government IT procurement policies must be modernized.

ħ **EXAMPLE:** Recovery: By Dec 2013, after 400+ bug fixes: Availability >90%, Response time <1s, Error rate <1%, Capacity ≈50,000 concurrent users. By April 2014: 7.1 million signed up! Good processes fixed what bad processes broke.

2.4 Why Do IT Projects Fail?

Root Cause	Explanation	Example
Requirements Problems	Unclear, incomplete, or changing requirements not managed well	HealthCare.gov specs changed until summer 2013
Lack of User Involvement	Disengaged customers lead to misaligned product	"How hard can it be?" attitude
Unrealistic Expectations	Over-optimistic "can-do" attitude at all levels	Promising features in 3 months that need 12
Lack of Planning	No plan = no control; perceived rush to get started	Starting to code before requirements are clear
Unclear Objectives	No clear "elevator statement" project vision	Team doesn't know what they're building or why

2.5 The 36 Classic Mistakes (AntiPatterns)

These are seductive — they seem like good decisions at the time but lead to disaster!

Category	Key Mistakes to Know
People-Related	Weak personnel; Unmotivated team; Heroics; Adding people to a late project (Brooks's Law!); Customer-developer friction; Wishful thinking; Noisy office
Process-Related	Optimistic schedules; Insufficient planning; Abandonment of plan under pressure; Wasted fuzzy front-end time; Shortchanged QA; Code-like-hell programming
Product-Related	Requirements gold-plating (adding unnecessary features); Feature creep; Developer gold-plating (pet projects); Push-me pull-me negotiation
Technology-Related	Silver-bullet syndrome (magical tool fix-all); Overestimating new tools; Switching tools mid-project; No source-code control

EXAM TRICK: BROOKS'S LAW: "Adding manpower to a late software project makes it

later." (People-Related Mistake #6) This is a classic exam question. New people need time to ramp up, and they distract existing members!

EXAMPLE: Feature Creep: A project to build a simple inventory system grows to include reporting, customer CRM, vendor portal, and AI analytics. Every added feature delays delivery. A good PM says NO to scope creep!

2.6 What Helps Projects Succeed?

The Standish Group's "Extreme CHAOS" (2001) identified 10 success factors:

#	Success Factor	Why It Matters
1	Executive Support	Leadership backing = resources and priority
2	User Involvement	Customer knows what they need — involve them!
3	Experienced Project Manager	Right person at the helm makes all the difference
4	Clear Business Objectives	Everyone knows WHY they're building this
5	Minimized Scope	Less scope = lower risk, faster delivery
6	Standard Software Infrastructure	Use proven tools and platforms
7	Firm Basic Requirements	Stable requirements = stable design
8	Formal Methodology	Follow a structured process consistently
9	Reliable Estimates	Realistic planning from the start
10	Other (small milestones, etc.)	Frequent checkpoints, competent staff

EXAM TRICK: In the exam, if asked for TOP 3 success factors: (1) Executive Support, (2) User Involvement, (3) Experienced PM. These are consistently #1, #2, #3 in Standish reports.

03 — SPM Fundamentals & Project Life Cycles

QUICK SUMMARY

- Formal PM brings real benefits: better control, lower costs, higher quality, shorter times.
- Boehm's Top 10 Metrics are fundamental laws every SPM student must know.
- PMI recognizes 9 Knowledge Areas — these are the domains a PM must master.
- Project Life Cycle: Predictive (Waterfall) vs. Iterative/Incremental vs. Adaptive (Agile).
- IT projects have TWO concurrent cycles: Project Life Cycle (PLC) + SDLC.

3.1 Benefits of Formal Project Management

Why Use Formal Project Management?
Better control of financial, physical, and human resources
Improved customer relations (clear expectations, regular communication)
Shorter development times (structured approach)
Lower costs (waste reduction through planning)
Higher quality and increased reliability
Higher profit margins
Improved productivity
Better internal coordination
Higher worker morale — less "death marches" and burnout!

3.2 Barry Boehm's Industrial Software Metrics — Top 10

These are empirical laws derived from massive data collection across software projects:

#	Metric	What It Means for You
1	Fix-after-delivery costs 100x more	Find bugs EARLY in design — testing late is extremely costly
2	Max 25% schedule compression	You can rush but only 25%. Below that, quality collapses.
3	$1 dev = $2 maintenance	Software maintenance costs TWICE the development cost
4	Cost $\propto$ Lines of Code (SLOC)	Larger code = higher cost; concise code is valuable
5	People drive productivity most	Hire the best — 10-to-1 productivity difference between developers!
6	Software/Hardware cost ratio growing	Software now dominates IT budgets (hardware is cheap)
7	Only 15% of effort is programming	Most work is planning, design, testing — NOT coding!

8	Systems cost 3x individual; system-of-systems 9x	Integration multiplies cost dramatically
9	Walkthroughs catch 60% of errors	Code reviews are extremely effective — use them!
10	80/20 rule: 20% contributors = 80% output	Identify and nurture your star contributors

EXAM TRICK: Boehm's most-tested metrics: #1 (100x cost), #2 (25% compression), #3 ($2 maintenance), #7 (only 15% coding). Memorize these four first!

EXAMPLE: If your team finds a bug after software delivery that costs $10,000 to fix, Boehm's Metric #1 says it would have cost just $100 to fix during early design! This is why proper design reviews pay off enormously.

3.3 PMI's 9 Knowledge Areas

A project manager must be competent in all 9 areas:

Knowledge Area	Core Question	Example Activity
Integration Management	How do all parts fit together?	Develop project charter; manage changes
Scope Management	What work needs to be done?	Create WBS; control scope creep
Time Management	How long will it take?	Develop schedule; identify critical path
Cost Management	How much will it cost?	Estimate costs; manage budget
Quality Management	How do we ensure quality?	Quality planning; QA and QC activities
Human Resource Mgmt	Who does the work?	Acquire, develop, and manage team
Communications Mgmt	Who needs what information?	Status reports; stakeholder updates
Risk Management	What can go wrong?	Risk identification; response planning
Procurement Management	What do we buy externally?	Vendor selection; contract management

EXAM TRICK: Mnemonic for 9 Knowledge Areas: "I'm So Tired, Can't Handle Communication, Risk is Possible" = Integration, Scope, Time, Cost, Human Resources, Communications, Risk, Procurement.

3.4 Project Life Cycles — The Big Three

Life Cycle Type	Key Characteristics
PREDICTIVE (Waterfall)	Scope fully defined upfront. Changes are tightly restricted. Sequential phases. Best for stable, well-understood requirements.

ITERATIVE & INCREMENTAL	Phases repeat; scope can evolve. Each iteration adds functionality. Risk managed continuously. Good balance of flexibility and structure.
ADAPTIVE (Agile)	Scope defined per phase only. High flexibility. Multiple iterations per phase (sprints). Best for uncertain, changing requirements.

3.5 The Waterfall SDLC

Waterfall Phases (in order)
1. Requirements → Define what to build
2. Analysis → Understand the problem in detail
3. Design → Plan how to build it
4. Construction → Actually code it
5. Quality Assurance (Testing) → Verify it works correctly
6. Deployment → Release to users

When to use Waterfall:

- Clear, unambiguous, and stable user requirements
- Familiar, proven technology
- Low complexity
- Adequate time available
- Stable schedule

Failure symptoms of Waterfall:

- Extended integration problems discovered late
- Late risk resolution (problems found too late to fix cheaply)
- Requirements-driven functional decomposition
- Focus on documents and review meetings rather than working software

EXAM TRICK: Waterfall was actually proposed by Royce as an example of WHAT NOT TO DO — but practitioners adopted it backwards! If the exam asks about Waterfall's origin: Winston Royce, 1970 paper, advocated ITERATIVE feedback but people took his example as the model!

3.6 Iterative SDLC — The 4 Phases

The iterative model defines four life-cycle phases:

Phase	Goal	Key Activities	Output
INCEPTION	Is the system viable?	Scope formulation; feasibility analysis; business case planning	Project concept / Idea
ELABORATION	Can we build it within constraints? (MOST CRITICAL)	Elaborate vision & architecture; establish construction process; baseline architecture ASAP	Architecture milestone
CONSTRUCTION	Build intermediate releases	Resource management; component development; testing; assess releases against acceptance criteria	Intermediate releases
TRANSITION	Roll out to customer	Deployment engineering; packaging; training; assess vs. acceptance criteria; plan next iteration	Final product

EXAM TRICK: ELABORATION is the MOST CRITICAL phase — know this for the exam! It's where the architecture is established and the team proves the system can be built. Failure here = project failure.

EXAMPLE: Solving a jigsaw puzzle is like iterative development — you don't go strictly top-to-bottom or bottom-to-top. You find edge pieces (architecture), fill in sections (construction), and gradually the picture emerges (transitions to product).

3.7 PLC vs. SDLC — Two Concurrent Cycles

IT Projects Have TWO Concurrent Life Cycles
PROJECT LIFE CYCLE (PLC): Encompasses ALL project activities. Directed toward achieving PROJECT requirements (on-time, on-budget, scope). Process Groups: Initiating → Planning → Executing → M&C → Closing.
SYSTEM/SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC): Directed toward achieving PRODUCT requirements (functionality, quality). Phases: Requirements → Design → Build → Test → Deploy.
KEY INSIGHT: Both must run TOGETHER — neither alone is sufficient. In practice, they should be so interwoven they cannot be distinguished!

Waterfall Cost Profile vs. Iterative Cost Profile		
	WATERFALL	ITERATIVE
Management:	5%	10%
Requirements:	5%	10%
Design:	10%	15%

Code & Unit Test:	30%	25%
Integration & Test:	40%	25%
Deployment:	5%	5%
Environment:	5%	10%
→ Waterfall puts 40% in Integration & Test (too late!)		
→ Iterative spreads the effort more evenly (better risk management)		

04 — Agile Project Management

QUICK SUMMARY
- Agile = iterative approach where each sprint produces working software.
- SCRUM is the most popular agile framework — know all its roles, events, and artifacts.
- Sprints are typically 2 weeks long.
- Agile embraces change; Waterfall resists it.
- Selection of methodology depends on: org culture, project size, requirements clarity.

4.1 What Is Agile?

AGILE	An iterative approach to planning and guiding project processes. Projects are completed in small sections called iterations or sprints. Insights from each sprint shape the next.

Agile Key Facts
• Scrum is the most popular agile framework.
• Each sprint = typically 2 weeks.
• Lean methodology: only as much process as NECESSARY.
• Agile project management is specialized — different from conventional PM.
• Each iteration reviewed by team AND client representatives.
• Insights from critique determine next steps.

4.2 Agile Roles

Role	Also Called	Responsibilities
Development Team	The builders	Programmers, testers, designers, writers — anyone with hands-on role in product creation
Product Owner	Customer Representative	Bridges gap between customer, stakeholders, and dev team. Owns product backlog.
Scrum Master	Project Facilitator	Supports dev team, clears roadblocks, keeps agile process consistent. NOT the PM.
Stakeholders	Interested parties	Anyone with interest in the project outcome
Agile Mentor	Agile Coach	Experienced agile practitioner who advises new teams

EXAM TRICK: Common exam trap: The SCRUM MASTER is NOT the same as the Project Manager! The Scrum Master facilitates the process and removes obstacles — but does NOT manage the project in the traditional sense.

4.3 Agile Events (The Scrum Ceremonies)

Event	When	Purpose & Key Details
Project Planning	Start of project	Create product vision statement and product roadmap. Can happen in as little as ONE DAY.
Release Planning	Before each release	Plan the next set of features to release to users.
Sprint	Ongoing (2 weeks)	Short development cycle producing potentially shippable functionality.
Sprint Planning	Start of each sprint	Team commits to sprint GOAL. Scrum team selects items from product backlog.
Daily Scrum	Every day, 15 minutes	3 questions: What did I complete yesterday? What will I complete today? Any blockers?
Sprint Review	End of each sprint	Dev team DEMONSTRATES working functionality completed in the sprint.
Sprint Retrospective	End of each sprint	Team discusses: What went well? What should change? How to implement changes?

EXAM TRICK: Remember: Daily Scrum = 15 minutes MAX (called "standup"). Sprint Review = DEMO to stakeholders. Sprint Retrospective = INTERNAL team improvement meeting. These are different and commonly confused in exams!

EXAMPLE: Daily Scrum in practice: Each morning, the dev team stands (no chairs = stays short!) and each person answers: "Yesterday I finished the login API. Today I'll work on password reset. No blockers." — 15 minutes, done.

4.4 Agile Artifacts

Artifact	Description	Update Frequency
Product Vision Statement	Elevator pitch — how product supports org strategy. Short, sharp, inspiring.	Once a year
Product Roadmap	High-level view of requirements with loose time frame.	Twice a year
Release Plan	High-level timetable for working software releases.	Per release cycle
Product Backlog	Full prioritized list of ALL project scope. Once first requirement exists, backlog exists!	Continuously
Sprint Backlog	Goal, user stories, and tasks for CURRENT sprint only.	Each sprint
Increment	Working product functionality at end of each sprint.	Each sprint

EXAM TRICK: The Product Backlog is the MASTER LIST of everything. The Sprint Backlog is a SUBSET of the Product Backlog — only what the team committed to for this sprint. Never confuse these two!

Agile Project Steps (in order)
1. Product Owner identifies the PRODUCT VISION.
2. Product Owner creates the PRODUCT ROADMAP.
3. Product Owner creates the RELEASE PLAN.
4. Product Owner + Scrum Master + Dev Team plan SPRINTS and start creating the product.
5. During each sprint, team holds DAILY SCRUMS.
6. Team holds SPRINT REVIEW (demo).
7. Team holds SPRINT RETROSPECTIVE (improvement).

4.5 Methodology Selection Guide

Which methodology to use depends on your situation:

Scenario	Best Methodology
Short time + shifting requirements	AGILE
Complex + short time schedule	ITERATIVE
Clear requirements + long schedule + command-driven team	WATERFALL
Reliable + complex + schedule variability	AGILE
Unfamiliar technology + short schedule + schedule variability	AGILE or ITERATIVE

Factor	Waterfall	Iterative	Agile
Requirements clarity	Clear & stable	Reasonably clear	Unclear / evolving
Technology	Familiar & proven	Unfamiliar / complex	Unfamiliar
Team style	Command-driven	Structured teams	Self-motivated
Complexity	Low	High	High
Time schedule	Adequate time	Short schedule	Short, variable
Risk tolerance	Risk-averse	Moderate	Risk-tolerant

EXAM TRICK: Guiding question for selection: "Are requirements clear and stable?" → YES = Waterfall. "Is the project complex with short timeline?" → YES = Iterative. "Are requirements uncertain and changing?" → YES = Agile.

05 — Process Management & PM Phases

Ǩ QUICK SUMMARY
• A PROCESS encapsulates an organization's successful recipes — it must be chosen, tailored, and assessed.
• ALL projects have a process; those with NO defined process end up thrashing (wasted work).
• 5 project constraints: Scope, Quality, Cost, Time, Resources — changing one affects others.
• 5 PM Phases: Initiate → Plan → Execute → Monitor & Control → Close.
• Communication is #1 skill for a PM — face-to-face is always richest!

5.1 What Is a Process?

PROCESS	A set of documented procedures, methods, practices, and tools used to produce a software product. It encapsulates an organization's experience as successful recipes.

What a Process Answers
WHAT to do? → Tasks/Activities
HOW to do it? → Procedure/Practice
WHEN to do it? → Sequence of activities
WHAT are the artifacts? → Input/Output documents

Process descriptions contain:

- Sequence of steps to be executed
- Who executes each step
- Entry/exit criteria for major steps
- Supporting guidelines, checklists, and templates

EXAM TRICK: "Projects with no specified process end up THRASHING." — Thrashing = unproductive, chaotic work that wastes time. Always choose or define a process at project start!

5.2 Choosing & Tailoring a Process

Two Options When Choosing a Process:

6. TAILOR the organizational process — used when team members are from the same group as before AND the last project was successful.
7. SPECIFY a new process — used when people come from different organizations using different processes.

5 Steps to Tailor an Organizational Process
1. Determine how your project DIFFERS from the typical organizational project.
2. Create two lists: activities needed from the org process + activities NOT needed.
3. PROPOSE changes to the organizational process.
4. Circulate the tailored process within team and key personnel for review.
5. Integrate changes and move quickly for closure.

Process Assessment — Questions to Answer Throughout the Project
• Were the tasks and activities EFFECTIVE?
• How much EFFORT did each task require?
• What tasks were performed but WEREN'T in the process spec?
• How did the products CHANGE over time?
• WHEN in the project did we spend effort doing what?

5.3 The Five Project Constraints

Every project operates under FIVE constraints. Changing one affects the others:

Constraint	Definition	Example
SCOPE	Defines project boundaries — what WILL and WON'T be done.	"The system will handle orders but NOT returns."
QUALITY	Product quality (deliverable) + Process quality (how well PM works).	System must have 99.9% uptime; process must have weekly reviews.
COST	The budget established for the project.	$500,000 total budget across 12 months.
TIME	The timeframe within which the project must be completed.	Must launch by December 31, 2025.
RESOURCES	People, equipment, facilities — limited availability.	5 developers, 2 servers, $50K/month budget.

EXAMPLE: Cost and Time are INVERSELY RELATED: If you need to finish faster (reduce time), you need more people or better tools (increase cost). You cannot compress time without consequences!

5.4 The Five Phases of Project Management

Phase	Also Called	Key Activities	Key Outputs
INITIATING	Scope/Define	Develop project charter; identify stakeholders; define scope & objectives.	Project charter, PM assigned, Constraints, Assumptions
PLANNING	Plan	Collect requirements; create	Project management plan,

		WBS; estimate costs; develop schedule; plan risks.	WBS, Schedule, Budget
EXECUTING	Execute/Launch	Direct work; manage team; perform QA; conduct procurements.	Deliverables, Team development, Procurements
MONITORING & CONTROLLING	Monitor/Track	Monitor progress vs. plan; manage changes; control scope, schedule, cost, quality.	Performance reports, Change requests, Updated plans
CLOSING	Close-out	Finalize all activities; get client acceptance; complete post-implementation audit; lessons learned.	Final project report, Lessons learned, Client sign-off

EXAM TRICK: Exam favorite: The CLOSING phase includes LESSONS LEARNED. This is how organizations get better over time. Also remember: closing involves both Administrative Closure AND Contract Close-out.

5.5 Communication — The PM's Most Critical Skill

Communication Channel Effectiveness (Richest to Least Rich)
1. People in a conference room with a WHITEBOARD ← RICHEST (most effective)
2. Video Conferencing (face-to-face, real-time)
3. Phone (voice, real-time but no visual)
4. Videotape (visual, but not real-time)
5. Email (written, asynchronous)
6. Paper ← LEAST RICH
Ỹ RULE: The more complex the issue, the richer the channel you need!

EXAM TRICK: "Communicate, communicate, and communicate!" The PM's golden rule. If in doubt about a communication medium for a complex problem, always choose the richest available channel.

06 — Initiating the Project: Charter, Stakeholders & Org Structures

Ǩ QUICK SUMMARY

- The PROJECT CHARTER formally authorizes a project and is the PM's mandate.
- The SOW (Statement of Work) defines the "boundary conditions" — what will and won't be done.
- SMART Goals: Specific, Measurable, Assignable, Realistic, Time-related.
- Stakeholders: Anyone who affects or is affected by the project.
- Three org structures: Functional, Projectized, Matrix (Weak/Balanced/Strong).

6.1 The Initiating Phase

Initiating Phase — Key Purposes
• Define a new project or new phase of an existing project.
• Identify initial project scope, stakeholders, and project manager.
• Align stakeholder expectations with project purpose.
• Give stakeholders visibility into scope and objectives.
• Set the VISION of the project: what needs to be accomplished.

Two main processes in Initiating:

8. DEVELOP PROJECT CHARTER — Justifies and formally authorizes the project. Documents stakeholder requirements. Creates the partnership between customer and supplier.
9. IDENTIFY STAKEHOLDERS — Identifies all people/organizations impacted. Documents their interests. Basis for stakeholder management strategy.

Concept Exploration Phase (Pre-Project):

- Collecting project ideas → Funneling process
- Project Justification: ROI, Cost-benefit analysis, Competitive analysis
- Initial planning and estimates
- Procurement: RFP process, vendor selection, contract management
- Gathering initial team, identifying project sponsor

6.2 The Project Charter

PROJECT CHARTER	A document that justifies and formally authorizes a project or project phase. Documents stakeholder initial requirements and expectations. Forms the basis for the customer-supplier partnership.

Project Charter Inputs
• Project Statement of Work (SOW)
• Business Case
• Agreements (contracts, MOUs)
• Enterprise Environmental Factors (org culture, regulations)
• Organizational Process Assets (past project info, templates)

Project Charter Typical Outline
OVERVIEW
• Business need (Problem/Opportunity)
• Objectives (Project goal)
• Method or approach
GENERAL SCOPE OF WORK
• Success criteria
ROUGH SCHEDULE & BUDGET
ROLES & RESPONSIBILITIES
ASSUMPTIONS, RISKS, OBSTACLES

Detailed Charter Content:

Charter Section	What It Contains	Example
Project Purpose/Justification	Why is this project being done?	"Increase e-commerce revenue by 25% by launching new mobile app"
Measurable Objectives & Success Criteria	Scope, Time, Cost, Quality goals with specific measures	"Site must load in <2 seconds; project complete by June 30, 2025"
High-level Requirements	Product capabilities — NOT detailed requirements!	"Customer can shop by brand OR product category" (NOT UI layout)
Assumptions & Constraints	Accepted truths + limitations	Assumption: Users have internet access. Constraint: Must use AWS hosting.
Scope/Boundaries	What's IN and what's OUT	"Site is one-stop wellness info. Will NOT include HR portal access."
High-level Risks	Major uncertainty areas	"Existing customers may resist migration to new site"
Summary Milestone	Key dates and deliverables	"Requirements: Apr 30; Go-live: Jun

Schedule		30"
Summary Budget (ROM)	Rough estimate ±35-100%	"Estimated $150K-$200K total; software 40%, HR 50%, hardware 10%"
Stakeholder List	Key people affected by/affecting project	CIO, Marketing Manager, Development Team Lead, End Users
Approval Requirements	Criteria for customer acceptance	"Must implement all 'must-have' user stories from initiation"

EXAM TRICK: HIGH-LEVEL requirements vs. DETAILED requirements: Charter has HIGH-LEVEL (e.g., "customer can shop by brand"). Anti-example of charter requirement: "Products displayed in 4-across grid with grey background" — that's too detailed for a charter!

ROM (Rough Order of Magnitude) estimates:

- Can be as broad as ±100% for very early estimates
- Typically range ±35% for initial estimates
- These are NOT commitments — just planning guides

6.3 Statement of Work (SOW)

SOW	A description of the work required for the project. Sets the "boundary conditions." Can be used in the final contract — so be careful, specific, and clear!

SOW Template (7 Sections)
I. Scope of Work — Describe work in detail. Specify hardware/software involved.
II. Location of Work — Where must the work be performed?
III. Period of Performance — Start/end dates, working hours, billing info.
IV. Deliverables Schedule — List specific deliverables with due dates.
V. Applicable Standards — Company or industry standards relevant to the work.
VI. Acceptance Criteria — How the buyer determines if work is acceptable.
VII. Special Requirements — Certifications, experience levels, travel, documentation, testing, support.

EXAM TRICK: SOW vs. Charter: The Charter is the WHY and WHAT (high-level). The SOW is the precise WHAT and HOW (detailed, contractual). SOW is done AFTER Charter approval ("after the Go decision"). SOW is used when contracting out work.

6.4 SMART Goals

SMART	A framework for writing well-defined project goals. Named after Doran's criteria.

Letter	Stands For	Meaning	Example
S	SPECIFIC	Be specific in targeting an objective	"Reduce login time" → NOT "improve performance"
M	MEASURABLE	Establish measurable indicators of progress	"Reduce login time by 50%" — measurable with %
A	ASSIGNABLE	Make the objective assignable to ONE person	"John will own the login performance improvement"
R	REALISTIC	Achievable with available resources	"50% improvement in 3 months with 2 devs" (not 99%)
T	TIME-RELATED	State when the objective can be achieved	"By March 31, 2025"

EXAMPLE: SMART Goal: "John will reduce the average login response time from 4 seconds to 2 seconds (50% improvement) by March 31, 2025, using the existing 2-developer team."
— Specific ✓, Measurable ✓, Assignable ✓, Realistic ✓, Time-related ✓

6.5 Stakeholders

STAKEHOLDER	An individual, group, or organization who may affect, be affected by, or perceive itself to be affected by a decision, activity, or outcome of a project.

Stakeholder Triad — The Three Most Important Roles
l FUNCTION REPRESENTATIVE (Business Analyst / Subject Matter Expert)
→ The "business person." Knows what the business needs.
m EXECUTIVE SPONSOR (Project Champion)
→ Project's visionary and champion. Provides resources and authority.
→ NOT the PM. Not "Santa Claus." Not the "Tech Guy."
n PROJECT MANAGER (The Linchpin)
→ Must be "multi-lingual" — speaks business, technical, and management.
→ Bridges all stakeholders.

Key stakeholder roles:

Role	Description
Customer	Person/organization that acquires the product
User	Person/organization that actually uses the product
Project Manager	Responsible for managing the entire project
Sponsor	Provides financial resources and executive support
Functional Managers	Manage within HR, accounting, procurement — don't deal with products directly
Operations Management	Manage core business: R&D, manufacturing, testing
Sellers/Vendors	External companies providing components/services
PMO	Project Management Office — centralized coordination of all projects

6.6 Organizational Structures

Structure	Description	PM Authority	Best For
FUNCTIONAL	Organized by department (Engineering, Marketing, Design). P&L from production.	Little or None	Stable, ongoing operations
PROJECTIZED	Organized by project (Project A, B, C). PM has P&L responsibility.	High to Almost Total	Defense, construction, avionics
MATRIX (Weak)	Functional-centric with project coordination. Staff split time.	Limited (part-time PM)	Organizations transitioning to project-focused
MATRIX (Balanced)	Equal weight to functional and project needs.	Low to Moderate	Moderate complexity projects
MATRIX (Strong)	Project-centric within functional structure. Full-time PM.	Moderate to High	Complex, cross-functional projects

Functional Organization	Projectized Organization
PROS: Clear authority, eliminates duplication, encourages specialization, clear career paths	PROS: Unity of command, effective intra-project communication
CONS: "Silos" reduce customer orientation, longer decision cycles, project leaders have little power	CONS: Duplication of facilities, difficult career paths for staff

EXAM TRICK: Matrix organization key problem: DUAL REPORTING — staff have TWO bosses (functional manager + project manager). This creates resource conflicts and priority disputes. In exams, this is the most commonly cited CON of matrix organizations.

07 — Project Planning, Scope Management & WBS

QUICK SUMMARY

- PLANNING = knowing where you should be so you can control where you are.
- WBS (Work Breakdown Structure) = hierarchical decomposition of ALL project work.
- 6 Criteria test if a WBS activity is complete: Measurable, Bounded, Deliverable, Estimated, Time-bounded, Independent.
- The 100% Rule: WBS must capture 100% of project scope — no more, no less!
- Requirements types: Business, Stakeholder, Functional, Nonfunctional, Transition, Project, Quality.

7.1 Why Plan?

Planning Is Essential to Control
"If you have no plan, you have no control."
An effective way to exert control:
→ Know where you ARE (current status)
→ Know where you're SUPPOSED TO BE (the plan)
→ Take CORRECTIVE ACTION when there's a difference
Analogy: A commercial airliner from Chicago to Tokyo. Without a flight plan, the plane is just guessing. With a plan, the pilot knows exactly what's off-course and can correct it.

Common reasons people don't plan (and why they're wrong):

- DON'T BELIEVE in planning → Management culture problem. Fix: Change the culture.
- FIND IT PAINFUL → Planning pain is front-loaded. No-plan pain is spread everywhere and much worse.
- NO TIME to plan → The more complex the project, the MORE you need a plan.

7.2 Primary Planning Steps

10. Identify project scope and objectives
11. Define and Record Requirements
12. Identify project organizational environment (analyze characteristics; identify team; define roles)
13. Identify project products and activities (create WBS; estimate effort; allocate resources; schedule milestones)
14. Develop Change Management Plan
15. Identify Risks and Define Risk Strategies
16. Review and communicate plan
17. Obtain Plan Approval
18. Conduct Kick-off Meeting

Planning encompasses: Scoping, Estimation, Schedule, Resources, Risk, and Control Strategy.

7.3 Key Planning Documents

Document	Purpose	Contains
Project Charter / SOW	Authorize and define the project	Problem, objectives, scope, rough budget, stakeholders
Software Project Management Plan (SPMP)	Master plan document	Project overview, deliverables, org structure, milestones, tech process, budget, schedule
Communications Management Plan	Information flow to all parties	Status meetings, stakeholder management, reporting
Risk Management Plan	How to handle uncertainties	Risk identification, analysis, response strategies
Software QA Plan (SQAP)	Ensure quality standards	Testing plans, review processes, quality metrics
Configuration Management Plan	Control changes to deliverables	Baselines, change control procedures, version control
Responsibility Assignment Matrix (RAM)	Who is responsible for what?	Activities mapped to team members/roles

EXAM TRICK: The SPMP (or SDP) is sometimes called the most important project document. It AGGREGATES all other plans. Think of it as the master project bible — everything else references back to it.

Time Allocation by Phase — The 40-20-40 Rule			
Specification (Planning) : Implementation (Coding) : Testing = 40% : 20% : 40%			
	Planning	Code & Unit Test	Integration & Test
Commercial:	25%	40%	35%
Internet:	55%	15%	30%
Real-time:	35%	25%	40%
Defense:	40%	20%	40%
→ Note: Only 15% of total effort is pure programming (Boehm Metric #7)!			

7.4 Scope Management

SCOPE	All the work required to complete the project successfully — and ONLY that work. No more, no less.

5 Steps of Scope Planning
1. DEFINE the scope → What is included and excluded?
2. Develop the detailed project SCOPE STATEMENT
3. Define and develop the WBS
4. VERIFY project scope (get stakeholder acceptance)
5. CONTROL project scope (manage changes)

Scope Management Plan components:

19. Process for preparing a detailed project scope statement
20. Process for creating the WBS from the scope statement
21. Process for maintaining and approving the WBS
22. Process for obtaining formal acceptance of completed deliverables
23. Process for controlling changes to the scope statement (linked to Integrated Change Control)

7.5 Types of Requirements

Type	Description	Example
Business Requirements	High-level needs of the entire organization	"Increase e-commerce purchases by 25%"
Stakeholder Requirements	Needs of a specific stakeholder	"Supply chain apps must interface with website"
Functional Requirements	Describe product BEHAVIOR	"Customer can shop by brand or product category"
Nonfunctional Requirements	Behavioral QUALITIES (performance, security, availability)	"Website available 99.99% of the time per year"
Transition Requirements	Temporary capabilities for migration	"Data entry clerks can use keyboard shortcuts from old system"
Project Requirements	Actions the project itself must meet	"Must use hybrid agile-waterfall methodology"
Quality Requirements	Validate successful completion of deliverables	"Test group finds product in <30 seconds"

EXAM TRICK: Functional vs. Nonfunctional: FUNCTIONAL = WHAT the system does (behaviors). NONFUNCTIONAL = HOW WELL it does it (performance, security, usability, reliability). Both are "Solution Requirements" in PMBOK 5.

7.6 The Work Breakdown Structure (WBS)

WBS	A hierarchical description of ALL work that must be done to complete the project as defined in the Project Overview Statement (POS). It captures 100% of project scope.

WBS Key Terms
ā ACTIVITY: A chunk of work. Parent node in the hierarchy.
Ā TASK: A smaller chunk of work. Child of an activity.
٩ MILESTONE: A task of zero duration. Marks completion of an activity/phase. Usually an external event.
Ì WORK PACKAGE: The lowest level of WBS. Where actual work takes place. Has a person, schedule, and budget assigned.

WBS Conventional Levels:

- Level 1: Project name (e.g., "E-Commerce Platform Rebuild")
- Level 2: Major subsystems (e.g., "User Authentication," "Product Catalog," "Checkout")
- Level 3: Components/tasks of subsystems (e.g., "Login," "Registration," "Password Reset")
- Level 4: Subcomponents/subtasks (e.g., "OAuth Integration," "Email Verification")
- Level 5: Work packages — actual work assigned to a person with schedule and budget

The 100% Rule — Most Important WBS Rule!
WBS includes 100% of the work defined by the project scope.
Applies at EVERY level: child items must sum to 100% of parent.
WBS must NOT include work outside the scope (no more than 100%).
WBS must NOT miss any work within scope (no less than 100%).
→ Example: If "User Management" is 30% of project, all its sub-tasks must together total 100% of User Management work.

7.7 The 6 Criteria for WBS Completeness

Every activity in the WBS must satisfy all SIX criteria to be considered properly decomposed:

Criterion	What It Means	Test Question
1. Measurable Status	Can you ask "how complete is this?" and get a real answer?	"I've written 150 of 300 pages → 50% complete" ✓
2. Bounded (Start/End Events)	Has a clearly defined starting trigger and ending signal	Start: "Final acceptance tests complete." End: "Customer approves docs." ✓
3. Deliverable	Produces a visible, tangible result (document, product, signature)	Approved design document, working module, signed form ✓
4. Cost/Time Estimate	Can estimate time and cost of completion	"3 weeks, 2 developers, $15K" ✓

5. Acceptable Duration	Duration fits within project's acceptable limits	Not so long you can't track it; not so short it's trivial ✓
6. Work Independence	Once started, can continue without interruption or additional inputs	No blocking dependencies that would halt mid-activity ✓

EXAM TRICK: Mnemonic for 6 WBS criteria: "My Big Dog Can't Ignore Wildlife" = Measurable, Bounded, Deliverable, Cost-estimated, (Time-limited), Independent, Work-independent. Or just memorize: STATUS, START/END, DELIVERABLE, COST/TIME, DURATION, INDEPENDENT.

7.8 Three Approaches to Building the WBS

Approach	Description & Example
NOUN-TYPE	Defines work in terms of COMPONENTS (physical or functional) that make up the deliverable. Best for product-focused projects. Example: Break "E-Commerce Site" into: Homepage, Product Pages, Cart, Checkout, User Account.
VERB-TYPE	Defines work in terms of ACTIONS that must be done. Best for process-focused projects. Example: Design → Build → Test → Implement each system feature.
ORGANIZATIONAL	Defines work in terms of ORGANIZATIONAL UNITS responsible. Best for large enterprises. Example: Engineering Department → Marketing Department → QA Department → Deployment Team.

EXAMPLE: There is NO single correct WBS! If you put 5 team members in separate rooms and asked each to build the WBS, you'd get 5 different but potentially all valid WBSs. The goal is completeness and clarity, not a specific structure.

7.9 Scheduling Tools

Tool	Description & Use
GANTT CHART	Bar chart showing project schedule. X-axis = time, Y-axis = tasks. Shows task durations and dependencies visually. Most common scheduling tool.
PERT (Program Evaluation and Review Technique)	Network diagram showing task dependencies. Uses optimistic, most likely, and pessimistic time estimates. Good for uncertain durations.
CPM (Critical Path Method)	Used with PERT to identify the longest path

	through the project. The critical path determines the minimum project duration. Delays on the critical path delay the whole project!
WBS in Microsoft Project	Move WBS into task sheet → Add resources → Add costs → Assign to tasks → Establish dependencies → Refine → Create baseline → Track progress.

EXAM TRICK: CRITICAL PATH = the longest sequence of dependent tasks = minimum project duration. Delays on the critical path directly delay project completion. Non-critical tasks have FLOAT/SLACK (they can be delayed without affecting the project end date).

WBS Is the Basis for Many Things
Network scheduling (PERT/CPM)
Cost estimation and budgeting
Risk analysis (identifies risky work packages)
Organizational structure definition
Progress monitoring and control
Performance measurement (earned value)

- Final Exam Quick Reference

Must-Know Definitions

Project	Temporary endeavor to create a unique product or service. Has scope, time, cost, and quality constraints.
Program	A collection of related projects. Larger in scope and longer in duration than individual projects.
WBS	Hierarchical breakdown of ALL project work. Must capture 100% of scope. Foundation of scheduling, costing, and risk analysis.
Critical Path	The longest path through the project network. Determines minimum project duration. Delays here = project delays.
Scrum	Agile framework with defined roles (Product Owner, Scrum Master, Dev Team), events (Sprint, Daily Scrum, Reviews, Retrospectives), and artifacts (Backlog, Increment).

Key Numbers to Memorize

Number	What It Refers To
100x	Cost to fix a bug after delivery vs. during design (Boehm #1)
25%	Maximum schedule compression possible (Boehm #2)
$2	Maintenance cost for every $1 spent on development (Boehm #3)
15%	Percentage of software effort that is actually programming (Boehm #7)
60%	Errors caught by walkthroughs/code reviews (Boehm #9)
80/20	Rule: 20% of contributors produce 80% of output (Boehm #10)
39%	CHAOS 2012: Percentage of successful IT projects
43%	CHAOS 2012: Percentage of challenged IT projects
18%	CHAOS 2012: Percentage of failed/cancelled IT projects
16.2%	CHAOS 1995: Success rate (original baseline — has improved since)
2 weeks	Typical sprint/iteration length in Agile/Scrum
15 minutes	Maximum daily scrum duration
5	Number of project constraints (Scope, Quality, Cost, Time, Resources)
5	Number of PM process groups (Initiate, Plan, Execute, Monitor, Close)
9	Number of PMI Knowledge Areas
6	WBS completeness criteria
4	Iterative SDLC phases (Inception, Elaboration, Construction, Transition)
4	Project 4 P's (People, Product, Process, Project)

Common Exam Traps

Watch Out For These!
◊ Scrum Master ≠ Project Manager (different roles, different responsibilities)
◊ Sprint Review ≠ Sprint Retrospective (Review = demo to stakeholders; Retrospective = internal improvement)
◊ Product Backlog ≠ Sprint Backlog (Product = all scope; Sprint = current sprint only)
◊ Charter has HIGH-LEVEL requirements; SOW has DETAILED requirements
◊ Waterfall was NOT designed to be used sequentially — Royce's original paper said the opposite!
◊ Adding people to a late project makes it LATER (Brooks's Law)
◊ Elaboration = most CRITICAL phase of iterative SDLC (not Construction!)
◊ WBS must be 100% — never more or less than total project scope
◊ Quality has TWO types: Product quality AND Process quality
◊ PLC and SDLC are TWO separate but concurrent life cycles in IT projects

EXAM TRICK: FINAL TIP: In any exam question about project failure, REQUIREMENTS problems are almost always the #1 root cause. The Standish CHAOS reports consistently rank requirement issues at the top of failure reasons. When in doubt, "requirements" is likely the right answer!

— Good Luck on Your Mid-Term! You've Got This! 🎓 —